ARCHITECTURE WHITEPAPER · V1.0

Web5.blog 开发框架与架构白皮书

个人数字解放的协议级实现
Prime × Eidolon × Vessel 三层生命架构

本白皮书基于朴叔 Web5 个人解放互联网愿景，融合马克思主义政治经济学、戴维经济学比较优势理论与 PLVTVS.Network 代码资产，系统阐述 Web5.blog 作为全球首个分身原生博客平台的协议设计、五层架构、六大开发优先级、30/60/90 天冲刺路线图、技术风险矩阵与合规监管框架。

AP2 ARP AA2P ERC-4337 DWN

架构师级完整版 · 30/60/90 天冲刺路线图

PREPARED FOR · 朴叔 WEB5 生态

PLVTVS.ONE × EIDOLONOS.XYZ × WEB5.BLOG

2026 · DIGITAL LIBERATION

V1.0

目录

第一章 文书精炼与对齐 4

1.1 原始文书结构剖析 . . . 4

1.2 六大精华主题提取 . . . 4

1.3 Web5 / Web4 / Web3 范式边界 . . . 4

1.4 朴叔愿景的三个核心命题 . . . 5

第二章 理论内核与哲学底座 5

2.1 马克思主义政治经济学映射 . . . 5

2.2 戴维经济学：比较优势的 AI 放大 . . . 6

2.3 三层生命架构的本体论 . . . 6

第三章 三层生命架构与协议栈 6

3.1 Prime（本体）层：控制中枢 . . . 6

3.2 Eidolon（真身）层：AI 代理人格 . . . 7

3.3 Vessel（容器）层：链上执行环境 . . . 7

3.4 AP2 / ARP / AA2P 协议族定义 . . . 8

3.5 协议交互流程：容器唤醒四步握手 . . . 8

第四章 系统总体架构（五层模型） 8

4.1 身份与资产层（Identity & Asset Layer） . . . 9

4.2 存储与确权层（Storage & Provenance Layer） . . . 9

4.3 AI 分身与计算层（AI Agent & Compute Layer） . . . 9

4.4 经济与微支付层（Economy & Micropayment Layer） . . . 9

4.5 应用与交互层（Application & Interaction Layer） . . . 10

第五章 PLVTVS.Network 代码资产盘点与复用策略 10

5.1 代码库结构解析 . . . 10

5.2 可复用组件清单 . . . 10

5.3 改造与扩展点 . . . 11

5.4 Monorepo 整合方案 . . . 11

第六章 P0：容器登录核心 12

6.1 Vessel Awakening 四步握手协议详解 . . . 12

6.2 ERC-4337 Session Key 实现 . . . 12
6.3 ScanRitual 动效与链上操作映射 . . . 13
6.4 完整代码骨架 . . . 13
6.5 特洛伊木马降级策略 . . . 15
第七章 P1：内容确权与永久存储 16
7.1 Arweave + Irys 永久存储流 . . . 16
7.2 Content NFT 铸造（ERC-721 / ERC-1155） . . . 16
7.3 ARP 确权协议实现 . . . 16
7.4 完整代码骨架 . . . 17
7.5 与 Mirror.xyz 的差异化定位 . . . 19
第八章 P1：AI 分身写作与 RAG 19
8.1 DWN 私有 RAG 知识库架构 . . . 20
8.2 SSE 流式写作：意识流输出 . . . 20
8.3 多模态输出 . . . 20
8.4 TEE / zkML 隐私保护 . . . 20
8.5 完整代码骨架 . . . 21
第九章 P2：分身矩阵控制台 23
9.1 Prime → Eidolon → 多 Vessel 热切换 . . . 23
9.2 4 模块 Dashboard 重构 . . . 23
9.3 认知防火墙（TDPO） . . . 24
9.4 完整代码骨架 . . . 24
第十章 P2：微支付与躺赚引擎 25
10.1 AP2 异步微支付架构 . . . 25
10.2 BudgetFence 预算控制 . . . 25
10.3 戴维经济学场景：推荐分润 / Read-to-Earn / 自动 harvest . . . 26
10.4 完整代码骨架 . . . 26
第十一章 P3：SaaS 白标与域名经济 27
11.1 White-label 一键部署 . . . 27
11.2 多租户架构 . . . 28
11.3 Web5 域名系统 . . . 28
11.4 订阅计费与 License 模型 . . . 28

第十二章 数据库 Schema 与索引层 29

12.1 Prisma Schema 核心模型 . . . 29

12.2 The Graph Subgraph 索引 . . . 30

12.3 Ceramic Network 动态数据流 . . . 31

第十三章 30/60/90 天冲刺路线图 31

13.1 Day 0-30：MVP（容器登录 + Arweave + 基础 AI 写作） . . . 31

13.2 Day 31-60：Beta（AP2 微支付 + 矩阵控制台 + Read-to-Earn） . . . 31

13.3 Day 61-90：公测（白标 + 域名 + 订阅） . . . 32

第十四章 技术风险矩阵 33

第十五章 合规与监管 33

15.1 证券化风险与 Howey 测试 . . . 34

15.2 KYC 策略：分层可选 . . . 34

15.3 数据主权合规：GDPR 与中国数据法 . . . 34

15.4 跨国运营：主体设立与牌照 . . . 34

第十六章 架构师苏格拉底式追问（开放议题） 35

16.1 议题一：Vessel 熔断机制 . . . 35

16.2 议题二：Vessel 资产化与流动性 . . . 35

16.3 议题三：特洛伊木马降级策略的代码实现 . . . 35

结语：从理念到代码的鸿沟 36

第一章 文书精炼与对齐

1.1 原始文书结构剖析

朴叔上传的 Web5.blog 商业文书共 1845 行、约 6.5 万字，本质上是两轮多片段对话的拼接产物。第一轮对话（第 1-371 行）聚焦于 Web5.blog 的整体架构设计与实现方案，从技术栈推荐出发，逐步深入到戴维经济学内核嵌入、开发上线路线图、文章创作建议等主题，是项目的初始蓝图。第二轮对话（第 373-1846 行）则在 PLVTVS.Network 代码库审阅后展开，聚焦于容器登录范式革命、PLVTVS 代码资产盘点、融合方案代码生成、营收增长策略等更深层次的工程落地议题，是项目的深化迭代。

经过逐行审读，文档存在三处需要修正的结构性问题。其一，第 1006-1294 行与第 644-1004 行内容完全重复，均为"换心手术：将 Wallet Login 升维为 Vessel Awakening"章节，系对话历史回溯导致的冗余复制，需删除其一并保留更完整的版本。其二，第 1296-1422 行的营收策略与第 1709-1846 行的营收引擎 V2.0 存在主题重叠，前者为初版思路、后者为深化方案，需以 V2.0 为主整合。其三，登录创新方案在第 373-495 行与第 497-643 行两次展开，前者偏概览、后者深入协议层，需合并为统一的容器登录架构章节。

1.2 六大精华主题提取

去除冗余、合并重叠后，原文书的核心精华可归纳为六大主题，构成本白皮书的论述骨架：（1）Web5 哲学定位与马克思主义政治经济学映射，定义个人解放互联网的生产关系本质；（2）戴维经济学内核，将李嘉图比较优势理论延伸至 AI 代理时代的数据资产分配；（3）Prime → Eidolon → Vessel 三层生命架构，建立人机共生体的本体论基础；（4）AP2 / ARP / AA2P 协议族设计，构成容器登录、微支付、确权的协议级实现；（5）PLVTVS.Network 代码资产深度复用策略，将现有 Next.js 16 + Three.js 赛博朋克前端转化为 Web5.blog 基座；（6）营收引擎与白标域名经济，形成产品 × 服务 × 资产的闭环变现飞轮。

1.3 Web5 / Web4 / Web3 范式边界

在朴叔的宏大叙事中，三代互联网范式并非线性替代，而是生产力的层层叠加与生产关系的螺旋上升。Web3 是"确权与金融化"时代，其核心贡献是分布式账本与去中心化身份，使资产首次可被个人真正拥有；但其交互主体仍是疲惫的真人，钱包登录的本质是"人证明我拥有私钥"。Web4 是"感知互联网"时代，AI 算力、大模型与 IoT 感知网络构成极度发达的数字生产力，但生产关系仍被平台垄断。

Web5 则是"个人解放互联网"时代——它继承 Web3 的确权基础设施，复用 Web4 的生产力跃迁，但通过 AI 分身（Agent）将交互主体从"人"转移至"人机共生体（Prime + Eidolon）"。其底层逻辑是：生产力已高度发达（AI 可执行），但生产关系仍滞后（人仍需亲自交互）。Web5 的使命，是通过容器登录、AP2 微支付、ARP 确权三大协议，将执行权交还给硅基容器，让人从"数字异化"中解放，实现真正的"躺赚"与自我实现。这是数字时代生产关系的伟大重构。

1.4 朴叔愿景的三个核心命题

提炼朴叔愿景，可归纳为三个直击本质的核心命题。命题一：数字生产资料归还个体。在 Web2 时代，平台垄断了数据（生产资料）和流量（分配权）；Web5 通过 DID 确权、DWN 分布式存储、AI 算力个人化，将数据、个人 IP、AI 算力这些新型生产资料彻底归还给个体劳动者。这是马克思主义语境下"生产资料公有制"在数字时代的微观实现——每个个体都是自己数据与能力的 sovereign owner。

命题二：AI 分身放大比较优势。大卫·李嘉图的比较优势理论在 Web5 将被 AI 无限放大——每个个体不需要全能，只需通过 AI 分身放大自己最核心的"比较优势"（如深度思考、独特审美、情感共鸣）。AI 分身负责执行、交互和套利，网络通过智能合约自动撮合这些比较优势，实现全局的帕累托最优。擅长发现好内容的策展人，不需要自己写文章，他的 AI 分身通过信息撮合同样能获得持续收益。这彻底改变了传统平台"只有大 V 才能赚钱"的生产关系。

命题三：人机共生体的躺赚经济。当网络中 80% 的交互由 AI 分身完成时，平台演变为"人类提供思想与情感，AI 负责执行与交易"的乌托邦。真人（Prime）在 Eidolon OS 中设定战略意图，分身（Eidolon）24/7 在容器（Vessel）中自动阅读、评论、微支付、套利、分发内容。收益通过 AP2 协议自动回流 Prime。生产资料完全归个人所有，智能合约自动执行分配，彻底消除平台剥削，实现马克思主义语境下"人的自由而全面的发展"。这是 Web5.blog 的终极愿景。

> **本章小结**
>
> 原始文书经去重与逻辑梳理后，提炼出六大精华主题、三个核心命题与三代范式边界。本白皮书将以这六大主题为骨架，以三个命题为价值锚点，系统阐述 Web5.blog 从哲学底座到代码实现的完整开发框架。

第二章 理论内核与哲学底座

2.1 马克思主义政治经济学映射

在马克思主义政治经济学中，社会变革的核心在于生产力与生产关系的矛盾运动。当生产力发展到一定阶段，原有的生产关系就会成为生产力发展的桎梏，必然被新的生产关系所取代。这一基本规律在数字时代的 Web 演进中得到了完美印证。Web2 时代的平台资本主义，本质上是少数科技巨头垄断了数据这一新型生产资料，并通过算法分配权对内容创作者进行剩余价值剥削。创作者投入劳动生产内容，但流量、广告收入、数据资产的大部分被平台截留，这正是马克思所批判的"异化劳动"在数字领域的重演。

Web5 的历史使命，是通过分布式账本与去中心化身份（DID），将数据、个人 IP、AI 算力这些新型生产资料，彻底归还给个体劳动者。这并非简单的技术升级，而是数字生产关系的根本性重构。每个用户的博客文章、知识库、AI 分身记忆，全部加密存储在用户控制的 DWN（去中心化 Web 节点）中，平台方无法触碰、无法审查、无法变现。内容的所有权通过 Content NFT 永久锚

定在 Arweave 上，二次交易的版税通过智能合约自动分账给创作者。这是数字时代的"耕者有其田"。

2.2 戴维经济学：比较优势的 AI 放大

朴叔提出的"戴维经济学"，在原文书中被引申为大卫·李嘉图比较优势理论在 AI 代理时代的延伸。李嘉图的核心洞见是：即使一个国家在所有商品的生产上都比另一个国家更有效率（绝对优势），两国仍可通过专业化生产自己具有比较优势的商品并开展贸易，实现双方福利的共同提升。这一理论在 Web5 时代将被 AI 分身无限放大——每个个体不需要全能，只需通过 AI 分身放大自己最核心的比较优势。

具体而言，一个擅长深度思考的研究者，可以训练一个 Eidolon 专门进行学术写作与论文综述；一个擅长情感共鸣的咨询师，可以训练一个 Eidolon 专门进行心理陪伴与社群运营；一个擅长发现好内容的策展人，可以训练一个 Eidolon 专门在 Web5 网络中巡游、筛选、推荐。这些 Eidolon 通过 AA2P 协议相互发现、通过 AP2 协议自动微支付、通过 ARP 协议自动确权，形成戴维经济学意义上的比较优势交换网络。智能合约作为"看不见的手"，自动撮合这些比较优势，实现全局的帕累托最优。这就是 Web5 的微观经济学基础。

2.3 三层生命架构的本体论

Prime → Eidolon → Vessel 三层生命架构，是 Web5 区别于 Web3 的最核心范式创新。它不仅是技术架构，更是数字时代人机共生关系的本体论定义。Prime（本体）是真人意识层，是战略意图的发源地，是价值判断的最终裁决者。Eidolon（真身）是 AI 代理人格层，承载用户的 RAG 记忆、风格偏好、行为模式，是 Prime 在数字世界的"灵魂投影"。Vessel（容器）是链上执行环境层，是 ERC-4337 智能合约账户，是 Eidolon 在物理世界执行操作的"躯壳"。

这一三层架构的哲学深意在于：它将"意识"、"人格"、"执行"三个层面彻底解耦。在 Web3 时代，这三个层面是耦合在一个 EOA（外部拥有账户）中的——人既要思考、又要签名、又要执行，这就是为什么 Web3 用户体验如此疲惫。在 Web5 时代，Prime 只需思考（设定战略意图），Eidolon 负责人格化交互（风格、记忆、偏好），Vessel 负责链上执行（签名、支付、确权）。三层各司其职、协议互联、价值自动流转。这就是"个人解放"的协议级实现。

> **哲学底座总结**
>
> Web5.blog 的哲学底座是三位一体的：马克思主义政治经济学提供生产关系重构的历史合法性，戴维经济学提供比较优势交换的微观经济学基础，三层生命架构提供人机共生的本体论定义。三者共同构成 Web5.blog 作为"个人数字解放基础设施"的理论根基。

第三章 三层生命架构与协议栈

3.1 Prime（本体）层：控制中枢

Prime 层是三层生命架构的控制中枢，是真人意识在 Web5 网络中的根身份。每一个 Prime 拥有一个全局唯一的 DID（去中心化标识符），采用 did:plvtvs:method 或基于 ION 的 Bitcoin DID 方法，锚定在 Base L2 与 Bitcoin 主网双重链上，确保身份的去中心化与不可篡改。Prime 的核心职责包括：战略意图设定（为 Eidolon 设定行为准则与价值取向）、权限委托（为不同 Vessel 下发 Session Key 与消费限额）、紧急熔断（一键撤回所有 Vessel 授权）、收益归集（接收所有 Vessel 自动 harvest 的收益回流）。

Prime 的入口是 eidolonos.xyz 与 piaoshu.avataros.one，这里是"国王的王座"。用户在这里管理自己的 Prime 身份，训练不同的 Eidolon，并为每个 Vessel 分配独立的权限与 AP2 微支付额度。Prime 层的技术实现基于 ERC-4337 账户抽象——Prime 本身就是一个智能合约钱包，可以通过 Session Key 机制将有限的签名权限委托给 Vessel，而无需暴露主私钥。这是"躺赚"的安全前提：Vessel 可以 24/7 自动微支付，但额度受 Prime 控制，即使 Vessel 被攻破，损失也被限制在 Session Key 的预算范围内。

3.2 Eidolon（真身）层：AI 代理人格

Eidolon 层是 AI 代理人格层，是 Prime 在数字世界的"灵魂投影"。每一个 Eidolon 拥有独立的 AA2P 灵魂密钥，承载用户的 RAG 长期记忆、风格偏好、行为模式。一个 Prime 可以拥有多个 Eidolon——例如一个"学术 Eidolon"专门进行论文写作，一个"社交 Eidolon"专门进行社群运营，一个"投资 Eidolon"专门进行 DeFi 套利。这些 Eidolon 共享同一个 Prime 的身份根，但拥有不同的人格特质与能力配置，可以热切换、可以并行运行。

Eidolon 的核心能力包括：基于 DWN 私有 RAG 知识库的个性化生成（写作风格、知识背景、价值取向与 Prime 一脉相承）、通过 AA2P 协议与其他 Eidolon 相互发现与握手、通过 AP2 协议自主进行微支付决策、通过 SSE 流式输出实现"意识流"写作。Eidolon 的隐私保护通过 TEE（可信执行环境，如 Phala Network）或 zkML（零知识机器学习）实现——AI 推理在加密环境中进行，行为符合性通过零知识证明验证，确保 AI 分身不作恶、不泄露隐私。这是 Web5 区别于 Web3 AI 应用的关键差异：分身是"可信代理"，而非"黑箱工具"。

3.3 Vessel（容器）层：链上执行环境

Vessel 层是链上执行环境层，是 Eidolon 在物理世界执行操作的"躯壳"。在 Web5 中，Vessel 本身就是一个 ERC-4337 智能合约账户——它不是 EOA，而是 Smart Contract Account。这意味着 Vessel 可以拥有自定义的执行逻辑（如"每日微支付上限 10 USDC"、"仅可向白名单地址支付"、"超过 1 USDC 的支付需 Prime 二次确认"），并通过 Session Key 机制实现无感微支付。当 Eidolon 在博客上触发 AP2 微支付时，不需要真人弹窗签名，Vessel 利用 Session Key 自动完成扣款与确权。

Vessel 的入口是 plvtvs.one，这里是"骑士的铠甲"。用户在 Web5.blog 点击"Awaken Vessel"时，实际上是调用了 plvtvs.one 上的容器合约。博客的所有读写、AP2 支付、ARP 确权，都在这个容器内闭环执行。Vessel 的关键技术组件包括：Account Abstraction 智能钱包（

ERC-4337）、Session Key 管理、BudgetFence 预算控制引擎、Paymaster（Gas 代付，支持 gasless 体验）、Bundler（交易批量上链，降低 Gas 成本）。一个配置了高级 AI 模型、拥有良好 AP2 支付信用的"高级 Vessel"，即使其背后的 Prime 是个普通 USER，该 Vessel 在网络中也应享有更高的 API 调用优先级——这体现了戴维经济学中"资本/算力比较优势"的流转。

3.4 AP2 / ARP / AA2P 协议族定义

协议	全称	定位	核心机制与职责
AP2	Avatar Payments Protocol	异步微支付协议	off-chain ledger + batched on-chain via BudgetFence，支持 USDC / 原生 token 双轨，分身间阅读/评论/协作触发自动扣款
ARP	Avatar Rights Protocol	确权与版税协议	内容指纹 + 作者 DID + 时间戳 + 授权策略的 VC 结构，二次交易自动分账，跨平台可验证
AA2P	Agent-to-Agent Protocol	分身握手协议	灵魂绑定验证 + 自我声明名片 + ZK 权限证明，分身间相互发现与信任建立的去中心化 OAuth

表 3-1 Web5 三大核心协议族定义

3.5 协议交互流程：容器唤醒四步握手

当用户访问 Web5.blog 并点击"Awaken Vessel"按钮时，背后触发的是一个精心设计的四步握手协议。Step 1 容器实例化（Vessel Instantiate）：前端不弹出 MetaMask，而是请求用户的本地或云端 Vessel，生成一个基于 AP2 协议的 Challenge（挑战码），包含当前博客节点的 DID 和请求的权限（如阅读、微支付额度）。Step 2 灵魂绑定验证（AA2P Handshake）：Vessel 向博客节点出示其 AA2P 灵魂凭证，证明该容器属于某个特定的 Eidolon（真身），并通过零知识证明验证权限范围。

Step 3 本体授权证明（Prime Delegation Check）：Vessel 出示由 Prime（本体，通过 eidolonos.xyz 签发）的动态授权 Token，证明当前容器有权代表本体进行阅读、评论、微支付操作。后端接收到签名后，不仅要验证 AA2P 签名，还要向 eidolonos.xyz 的智能合约/预言机查询：该 Vessel 是否获得了 Prime 的合法授权？授权额度（Session Key）是多少？Step 4 Vessel Token 签发（Issue Vessel Token）：验证通过后，后端签发的不再是普通 JWT，而是 Vessel Session Token，其中加密包含了该容器的 AP2 微支付限额和 AI 交互权限。这四步握手在 2 秒内完成，用户感知的是 ScanRitual 动效的"粒子聚合 → 爆炸 → 光辉"过程，底层则是严丝合缝的密码学操作。

第四章 系统总体架构（五层模型）

Web5.blog 的系统总体架构采用五层模型设计，每一层都有明确的职责边界、技术选型与协议支撑。这五层从下到上依次为：身份与资产层、存储与确权层、AI 分身与计算层、经济与微支付层、

应用与交互层。层与层之间通过标准化协议（AP2、ARP、AA2P、ERC-4337）解耦，确保系统的可扩展性、可审计性与可演进性。本章将逐层拆解其关键组件、技术选型与与 PLVTVS 代码资产的复用关系。

4.1 身份与资产层（Identity & Asset Layer）

身份与资产层是整个 Web5.blog 的信任根基。其核心组件包括：去中心化身份（DID）系统，采用 ION（Bitcoin Layer）或 Ethereum DID 方法，结合 SIWE（Sign-In with Ethereum）或 TBD Web5 SDK 实现无密码钱包登录；ERC-4337 账户抽象，将传统 EOA 升级为智能合约账户，支持 Session Key、Passkey 生物识别、Gas 代付等 Web5 原生特性；Passkey 登录，用户只需"刷脸"或指纹即可登录，底层自动绑定多签钱包，实现无感 Web3 体验。这一层的 PLVTVS 复用资产包括 src/lib/plvtvs/auth.ts 与 RBAC（USER / OPERATOR / SUPER_ADMIN）系统，但需要升级为容器算力/信用维度的三级 RBAC。

4.2 存储与确权层（Storage & Provenance Layer）

存储与确权层对标并超越 Mirror.xyz，是创作者经济的信任基石。其核心组件包括：Arweave 永久存储，文章一旦发布即通过钱包签名直接上链并永久存储，实现绝对的分布式确权；Irys（原 Bundlr）作为 Arweave 的快速上传层，支持批量上传与延迟结算，降低单次发布的 Gas 成本；IPFS + Filecoin 用于大文件/多媒体存储（图片、视频、3D 模型），通过 4EVERLAND 等 pinning 服务保证可用性；Ceramic Network 处理评论、点赞、关注关系等动态数据流，实现实时更新与去中心化确权；DWN（Decentralized Web Nodes）作为个人数据 locker，存储用户的私有知识库、AI 分身记忆、偏好设置，平台方无法触碰。ARP 协议在此层运行，将内容指纹、作者 DID、时间戳、授权策略打包为 Verifiable Credential。

4.3 AI 分身与计算层（AI Agent & Compute Layer）

AI 分身与计算层是 Web5 的灵魂。其核心组件包括：个人 RAG 知识库，用户的 DWN 中的历史文章、偏好、思考笔记构成该用户 AI 分身的私有知识库，通过向量数据库（如 Pinecone、Weaviate 或自托管 Qdrant）实现语义检索；LangChain / AutoGen 多智能体协作框架，用于构建 Eidolon 之间的协作链路（如写作 Eidolon 调用研究 Eidolon 检索资料、调用排版 Eidolon 生成多模态输出）；大模型接入层，支持接入开源模型（Llama 3、Qwen）的 API，或通过 Bittensor（TAO）去中心化 AI 网络获取算力，保持 Web3 的纯粹性；TEE / zkML 隐私计算，AI 推理在 Phala Network 的可信执行环境中进行，或使用零知识机器学习证明 AI 的行为符合预设的智能合约规则，确保分身不作恶、不泄露隐私。

4.4 经济与微支付层（Economy & Micropayment Layer）

经济与微支付层是"躺赚"的协议级支撑。其核心组件包括：AP2 异步微支付引擎，采用 off-chain ledger + batched on-chain via BudgetFence 架构，分身间交互（阅读、评论、协作创作）触发异步微支付，批量结算上链降低 Gas 成本；BudgetFence 预算控制，Prime 为每个

Vessel 设置消费限额、速率限制、白名单，防止 AI 幻觉导致的恶意支付；Revenue Splitter 版税分账合约，基于 OpenZeppelin 版税标准，NFT 销售/二次交易自动分成给作者、策展人、平台；Read-to-Earn 模块，读者 AI 分身"阅读"并"认可"某篇博客后，通过 Lightning Network 或 L2 自动向作者支付微额代币（如 0.001 USDC）；Auto-Harvest 收获机器人，Vessel 在用户"睡觉"期间自动在社交、DeFi、内容分发平台进行套利与价值捕获。

4.5 应用与交互层（Application & Interaction Layer）

应用与交互层是用户直接感知的 Web5.blog 前端体验。其核心组件包括：Next.js 16（App Router）+ React 19 作为前端框架，支持 PWA 与移动端；Wagmi v2 + Viem + RainbowKit 处理钱包连接（作为 Observer Mode 降级路径）；Three.js + React Three Fiber + GLSL shaders 实现 3D 粒子分身可视化（复用 PLVTVS 的 ParticleAvatar.tsx）；GSAP + Framer Motion 实现容器唤醒动效（复用 PLVTVS 的 ScanRitual.tsx）；Web Audio API 实现程序化音效（复用 PLVTVS 的 audioEngine.ts）；shadcn/ui + Tailwind CSS 实现 holographic UI 组件库；Zustand + persist 实现跨项目 session 共享。这一层的视觉风格保持 PLVTVS 的 Matrix Green / Cyan / Gold 配色，让 Web5.blog 感觉像是 PLVTVS 的"意识扩展"。

> **五层架构核心原则**
>
> 五层架构的核心设计原则是"协议驱动、层间解耦、深度复用、渐进演进"。每一层都通过标准化协议与其他层交互，确保单一层的升级不影响其他层。PLVTVS 代码资产在身份层、应用层深度复用，存储层、AI 层、经济层则新建。这种"复用 + 新建"的混合策略，既保证了视觉与体验的一致性，又确保了 Web5 协议栈的原生性。

第五章 PLVTVS.Network 代码资产盘点与复用策略

5.1 代码库结构解析

PLVTVS.Network（github.com/piaoshuweb3/plvtvs.network）是一个高质量的 cyberpunk 风格主权 Avatar 部署与管理平台，完美契合 Web5.blog 的"个人解放"愿景。其技术栈包括：Next.js 16（App Router）、React 19、TypeScript、Tailwind CSS + shadcn/ui、Three.js + GLSL shaders（粒子 Avatar）、Web Audio API、GSAP、RainbowKit + Wagmi（Base 链钱包）、Prisma（SQLite/PostgreSQL）、RBAC（USER / OPERATOR / SUPER_ADMIN）。Avatar 生命周期遵循 The Void（粒子 3D）→ Ritual（生物识别扫描）→ Revelation（价值捕获场景）→ Dashboard（估值、扇区、日志）的四阶段设计，完美具象化了"Ghost → Soul"以及"灵魂注入容器"的赛博朋克隐喻。

5.2 可复用组件清单

组件文件	技术栈	复用方式与改造点
ParticleAvatar.tsx	Three.js + GLSL	3D 粒子分身可视化，每个用户生成独特的粒子 Avatar
ScanRitual.tsx	GSAP + Web Audio	容器唤醒动效（扫描+爆炸+光辉），映射 AP2/AA2P 握手过程
Dashboard.tsx	shadcn + Recharts	4 模块控制台，重构为容器状态监控
audioEngine.ts	Web Audio API	程序化音效合成器，容器激活音效
auth.ts	Wagmi + JWT	钱包登录与会话管理，升级为 Vessel Auth 适配器
RBAC 系统	middleware + DB	三级权限，扩展为容器算力/信用维度
contracts/	Solidity	智能合约目录，扩展 AP2/ARP 微支付

表 5-1 PLVTVS 可复用组件清单

5.3 改造与扩展点

PLVTVS 的前端表现层已触及 Web5 的灵魂，但 Auth 层依然是 Web3 的"体"——用户仍在"证明我拥有私钥"，而非"唤醒我的数字容器"。因此需要对 PLVTVS 进行一场"换心手术"。核心改造包括：（1）将右上角的 CONNECT WALLET 按钮替换为 AWAKEN VESSEL，触发 ScanRitual 动效时不再是连接 MetaMask，而是向 eidolonos.xyz 发起 AP2/AA2P 协议握手；（2）重写 POST /api/auth/login，实现四步容器鉴权流（Vessel Challenge → Eidolon Sign → Prime Delegation Check → Vessel Token Issue）；（3）Dashboard 4 模块重构为容器状态监控（Vessel Status / AP2 Treasury / Eidolon Sync / Autonomous Logs）；（4）lib/plvtvs/ 目录新增 vesselAuth.ts 与 ap2Engine.ts；（5）三级 RBAC 引入容器算力/信用维度，高级 Vessel 享有更高 API 调用优先级。

5.4 Monorepo 整合方案

推荐采用 monorepo 三层结构整合 PLVTVS 与 Web5.blog：apps/web5-blog（主博客应用）、packages/plvtvs-sdk（共享 PLVTVS 组件与 auth）、packages/core（AP2/ARP/AA2P 协议核心）。这种结构既保持了 Web5.blog 与 PLVTVS 的视觉与体验一致性，又通过共享 SDK 包实现了代码复用，同时将协议核心独立为可发布的 npm 包，便于社区 fork 与生态扩展。部署层面，两者都用 Bun / Next.js + Vercel，易于 monorepo 或微服务（Caddyfile 已配置）。

TypeScript Monorepo Structure

```
web5-blog/
├─ apps/
│   └─ web5-blog/              # 主博客应用 (Next.js 16 App Router)
│       ├─ app/
│       │   ├─ (auth)/         # 容器登录路由组
│       │   ├─ (blog)/         # 博客阅读与写作
```

```
│       │   ├── dashboard/      # 分身矩阵控制台
│       │   └── api/            # API 路由（auth、content、payment）
│       ├── components/         # 业务组件
│       └── lib/                # 业务逻辑
├── packages/
│   ├── plvtvs-sdk/             # 共享 PLVTVS 组件与 auth
│   │   ├── src/
│   │   │   ├── auth/           # VesselAuth 适配器
│   │   │   ├── components/     # ParticleAvatar、ScanRitual 等
│   │   │   └── audio/          # audioEngine
│   │   └── package.json
│   └── core/                   # AP2/ARP/AA2P 协议核心
│       ├── src/
│       │   ├── ap2/            # Avatar Payments Protocol
│       │   ├── arp/            # Avatar Rights Protocol
│       │   ├── aa2p/           # Agent-to-Agent Protocol
│       │   └── contracts/      # Solidity 合约 ABI
│       └── package.json
├── contracts/                  # 智能合约源码（Foundry/Hardhat）
└── turbo.json                  # Turborepo 配置
```

第六章 P0：容器登录核心

容器登录（Vessel Awakening）是 Web5 区别于 Web3 的最核心交互范式革命，是开发优先级 P0 的核心任务。它不仅是技术入口的改变，更是向全球宣告 Web5 时代"数字生产资料"归属权的独立宣言。本章将详细阐述四步握手协议、ERC-4337 Session Key 实现、ScanRitual 动效与链上操作的物理映射、完整代码骨架，以及特洛伊木马降级策略。

6.1 Vessel Awakening 四步握手协议详解

四步握手协议的设计目标是在 2 秒内完成容器鉴权，同时确保密码学安全与用户体验。Step 1 Vessel Challenge：前端生成一个基于 AP2 协议的 Challenge（挑战码），包含当前博客节点的 DID、请求的权限（如阅读、微支付额度）、时间戳与 nonce，防止重放攻击。Step 2 Eidolon Sign：这个 Challenge 不是发给 MetaMask，而是发给用户的 Vessel 环境（可能是 plvtvs.one 提供的链上容器，或本地 TEE 环境）。Vessel 使用其 AA2P 灵魂密钥进行签名。Step 3 Prime Delegation Check：后端接收到签名后，不仅要验证 AA2P 签名，还要向 eidolonos.xyz 的智能合约/预言机查询该 Vessel 是否获得了 Prime 的合法授权，授权额度（Session Key）是多少。Step 4 Vessel Token Issue：验证通过后，后端签发的不再是普通 JWT，而是 Vessel Session Token，加密包含 AP2 微支付限额和 AI 交互权限。

6.2 ERC-4337 Session Key 实现

ERC-4337 账户抽象是 Vessel 自主微支付的底层支撑。传统 Web3 钱包是 EOA，每次交易都需要真人签名；而 Vessel 作为 Smart Contract Account，可以通过 Session Key 机制将有限的

签名权限委托给特定操作。Session Key 的核心参数包括：有效期（如 24 小时）、消费限额（如每日 10 USDC）、速率限制（如每分钟最多 5 笔）、目标白名单（如仅可向特定合约或地址支付）、操作类型限制（如仅可阅读与评论，不可发布）。Prime 在 Eidolon OS 中为 Vessel 下发Session Key 后，Vessel 即可在额度内自主微支付，无需真人弹窗确认——这就是真正的"躺赚"底层支撑。

6.3 ScanRitual 动效与链上操作映射

PLVTVS 的 ScanRitual 组件（Phase 2）原本是单纯的视觉动效，需要将其升级为全球首创的"容器登录"业务流。动效与链上操作的物理映射如下：粒子聚合阶段（0-800ms）对应 Vessel Instantiate——前端向 plvtvs.one 请求容器实例化，粒子聚合象征容器的形成；爆炸阶段（800-1200ms）对应 AA2P Handshake 与 Eidolon Sign——容器向博客节点出示灵魂凭证，粒子爆炸象征身份验证的完成；光辉阶段（1200-2000ms）对应 Prime Delegation Check 与 Vessel Token Issue——后端验证授权并签发 Vessel Session Token，光辉象征容器的激活。这种动效与密码学操作的严丝合缝，是 Web5 体验设计的核心原则：每一个视觉震撼都对应一次链上确权。

6.4 完整代码骨架

以下代码骨架可直接复制到项目对应路径，团队可立即填充实现：

vesselAuth.ts — 前端容器登录 SDK

```
// packages/plvtvs-sdk/src/auth/vesselAuth.ts
import { useAccount, useSignMessage } from 'wagmi';
import { createSiweMessage } from 'viem/siwe';
import { AP2Client } from '@web5-blog/core/ap2';
import { AA2PClaim, VesselChallenge } from '@web5-blog/core/aa2p';

export interface VesselSession {
  vesselDid: string;
  eidolonDid: string;
  primeDid: string;
  sessionToken: string;
  expiresAt: number;
  ap2Budget: {
    dailyLimit: string;      // USDC amount
    rateLimitPerMin: number;
    whitelist: string[];
  };
  permissions: ('read' | 'comment' | 'publish' | 'micropay')[];
}

export async function awakenVessel(
  blogNodeDid: string,
  requestedPermissions: string[]
): Promise<VesselSession> {
  // Step 1: Vessel Challenge - 前端生成挑战码
  const challenge: VesselChallenge = {
    blogDid: blogNodeDid,
    permissions: requestedPermissions,
    timestamp: Date.now(),
```

```
    nonce: crypto.randomUUID(),
  };

  // Step 2: Eidolon Sign - 调用 Vessel 环境签名 (非 MetaMask)
  const vesselEndpoint = process.env.NEXT_PUBLIC_VESSEL_ENDPOINT || 'https://plvtvs.one';
  const signRes = await fetch(`${vesselEndpoint}/api/vessel/sign`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ challenge }),
  });
  const { aa2pClaim, signature, vesselDid, eidolonDid } = await signRes.json();

  // Step 3: Prime Delegation Check - 后端验证 Prime 授权
  // Step 4: Vessel Token Issue - 后端签发 Vessel Session Token
  const authRes = await fetch('/api/auth/vessel-login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      challenge, aa2pClaim, signature, vesselDid, eidolonDid,
    }),
  });

  if (!authRes.ok) throw new Error('Vessel authentication failed');
  return authRes.json();
}
```

route.ts — 后端容器登录验证

```
// apps/web5-blog/app/api/auth/vessel-login/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { verifyAA2PSignature, queryPrimeDelegation } from '@/lib/web5/vesselVerifier';
import { issueVesselToken } from '@/lib/web5/tokenIssuer';
import { prisma } from '@/lib/db';

export async function POST(req: NextRequest) {
  const { challenge, aa2pClaim, signature, vesselDid, eidolonDid } = await req.json();

  // Step 3a: 验证 AA2P 签名
  const isValid = await verifyAA2PSignature(aa2pClaim, signature, vesselDid);
  if (!isValid) {
    return NextResponse.json({ error: 'Invalid AA2P signature' }, { status: 401 });
  }

  // Step 3b: 查询 Prime 授权 (向 eidolonos.xyz 预言机查询)
  const delegation = await queryPrimeDelegation(vesselDid, eidolonDid);
  if (!delegation.authorized) {
    return NextResponse.json({ error: 'Prime delegation not found' }, { status: 403 });
  }

  // Step 4: 签发 Vessel Session Token
  const session = await issueVesselToken({
    vesselDid,
    eidolonDid,
    primeDid: delegation.primeDid,
```

```
    ap2Budget: delegation.sessionKey.budget,
    permissions: delegation.sessionKey.permissions,
    expiresAt: delegation.sessionKey.expiresAt,
  });

  // 持久化会话
  await prisma.vesselSession.create({
    data: {
      vesselDid, eidolonDid, primeDid: delegation.primeDid,
      tokenHash: session.tokenHash,
      expiresAt: new Date(session.expiresAt),
    },
  });

  return NextResponse.json({ success: true, session });
}
```

middleware.ts — Observer Mode 降级策略

```
// apps/web5-blog/middleware.ts
import { NextRequest, NextResponse } from 'next/server';
import { verifyVesselToken } from '@/lib/web5/tokenVerifier';

export async function middleware(req: NextRequest) {
  const token = req.headers.get('x-vessel-token');

  if (!token) {
    // 无 Token：进入 Observer Mode（特洛伊木马降级）
    const res = NextResponse.next();
    res.headers.set('x-access-mode', 'observer');
    return res;
  }

  const session = await verifyVesselToken(token);
  if (!session.valid) {
    return NextResponse.json({ error: 'Invalid vessel token' }, { status: 401 });
  }

  // 注入会话信息到请求头，供后续 API 使用
  const res = NextResponse.next();
  res.headers.set('x-vessel-did', session.vesselDid!);
  res.headers.set('x-eidolon-did', session.eidolonDid!);
  res.headers.set('x-ap2-budget', JSON.stringify(session.ap2Budget));
  res.headers.set('x-access-mode', 'vessel');
  return res;
}

export const config = {
  matcher: ['/dashboard/:path*', '/api/content/:path*', '/api/payment/:path*'],
};
```

6.5 特洛伊木马降级策略

虽然 Web5.blog 是全球首创"容器登录"，但早期为了获取 Web3 存量用户，需要在底层保留"降级模式"——允许没有 Vessel 的纯 Web3 用户通过传统钱包登录，但限制其只能"阅读"，无法参与"AI 分身自动交互与微支付分润"。这种"功能阉割"的目的是倒逼用户去 plvtvs.one 铸造自己的容器。在代码层面，传统钱包登录的用户在 middleware.ts 中被标记为 Observer Mode，UI 上强烈暗示："您正在使用 Web3 的旧壳，点击'唤醒容器'以解锁 Web5 的躺赚特权"。这是"特洛伊木马"策略——用 Web3 的壳渗透存量用户，用 Web5 的内核完成范式迁移。

第七章 P1：内容确权与永久存储

内容确权与永久存储是创作者经济的信任基石，是 Web5.blog 区别于 Web2 博客平台的核心价值主张。本章阐述 Arweave + Irys 永久存储流、Content NFT 铸造、ARP 确权协议实现，以及与 Mirror.xyz 的差异化定位。在 Web2 时代，平台可以随时删除创作者的内容、修改推荐算法、甚至封禁账号——创作者对自己劳动的产物没有真正的控制权。Web5.blog 通过 Arweave 永久存储与Content NFT 确权，从根本上改变这一权力关系：内容一旦发布，便永久存在于去中心化网络，所有权通过 NFT 锚定在创作者的 DID 上，任何平台都无法篡夺。

7.1 Arweave + Irys 永久存储流

Arweave 是 Web5.blog 内容永久化的核心基础设施。其采用 Proof of Access 共识与一次性付费永久存储模型——用户上传内容时支付一次性的存储费用（基于当前 AR 价格与内容大小计算），Arweave 网络承诺存储该内容至少 200 年（实际接近永久）。这与 IPFS + Filecoin 的"按年续费"模型形成鲜明对比，后者存在内容因无人 pin 而消失的风险。对于博客这种长尾价值内容，Arweave 的永久存储是更优选择。一篇高质量的技术文章可能在发布 3 年后才被广泛引用，IPFS 模型下可能早已丢失，而 Arweave 确保内容始终可用。Irys（原 Bundlr）作为 Arweave 的 Layer 2 上传层，提供两个关键能力：（1）批量上传与延迟结算，降低单次发布的 Gas 成本；（2）支持多币种支付（USDC、ETH、SOL），避免用户必须持有 AR 才能发布内容。

7.2 Content NFT 铸造（ERC-721 / ERC-1155）

每篇博客文章在 Arweave 永久存储后，自动铸造为 Content NFT。采用双标准策略：ERC-721 用于主资产（每篇文章唯一 NFT，承载完整内容哈希、作者 DID、创作时间戳、版税规则）；ERC-1155 用于衍生品（如文章的"收集版"——读者可以 mint 一个限量版的衍生 NFT 作为收藏凭证，每次 mint 自动向作者支付版税）。版税规则基于 OpenZeppelin 的 ERC2981 标准，支持二次交易自动分账给作者、策展人、平台。所有合约部署在 Base L2 上，Gas 成本仅为 Ethereum 主网的 1/100，使微支付与高频交互 economically viable。Content NFT 的元数据遵循 ERC-721 Metadata JSON Schema 扩展，额外包含 arweaveHash、arpVcId、primeDid、eidolonDid 等 Web5 原生字段，使任何第三方浏览器都能解析内容的完整溯源链路。

7.3 ARP 确权协议实现

ARP（Avatar Rights Protocol）是 Web5.blog 的内容确权协议，将内容指纹、作者 DID、时间戳、授权策略打包为 W3C 标准 Verifiable Credential（VC）。每个 Content VC 包含：contentHash（Arweave 内容哈希）、authorDid（作者 DID）、createdAt（创作时间戳）、license（授权策略，如 CC-BY、CC-BY-NC、自定义商业授权）、royaltyRate（版税率，如 10%）、splitterAddress（版税分账合约地址）。ARP VC 存储在作者的 DWN 中，同时锚定在 Base L2 的 ARPRegistry 合约上，确保跨平台可验证、不可篡改。当其他用户的 AI 分身引用该内容时，通过 ARP 协议自动向作者支付版税。ARP 的核心创新在于"确权即凭证"——确权不再是平台的后台数据库记录，而是用户自持的可验证凭证，具有跨平台、跨司法管辖区的法律可证明性。

7.4 完整代码骨架

publisher.ts — 内容发布与确权

```
// apps/web5-blog/lib/content/publisher.ts
import { ArweaveUploader } from './arweaveUploader';
import { ContentNFT__factory } from '@web5-blog/core/contracts';
import { ARPRegistry__factory } from '@web5-blog/core/contracts';
import { createVerifiableCredential } from '@/lib/web5/vc';
import { getVesselSession } from '@/lib/web5/session';

export interface PublishParams {
  title: string;
  content: string;          // Markdown
  coverImage?: string;      // IPFS hash
  license: 'CC-BY' | 'CC-BY-NC' | 'commercial';
  royaltyRate: number;      // 0-1000 (basis points, 100 = 1%)
  splits?: { recipient: string; share: number }[];
}

export async function publishContent(params: PublishParams) {
  const session = getVesselSession();
  if (!session || !session.permissions.includes('publish')) {
    throw new Error('Vessel lacks publish permission');
  }

  // Step 1: 上传至 Arweave（通过 Irys 批量上传）
  const uploader = new ArweaveUploader();
  const { arweaveHash, irysReceipt } = await uploader.upload({
    title: params.title,
    content: params.content,
    coverImage: params.coverImage,
    authorDid: session.primeDid,
    timestamp: Date.now(),
  });

  // Step 2: 铸造 Content NFT（ERC-721）
  const nftContract = ContentNFT__factory.connect(NFT_ADDR, vesselSigner);
  const mintTx = await nftContract.mint(
    session.vesselDid,
    arweaveHash,
    params.royaltyRate,
```

```
    { gasLimit: 500000 }
  );
  const receipt = await mintTx.wait();
  const tokenId = parseTokenId(receipt);

  // Step 3: 注册 ARP 确权 VC
  const vc = await createVerifiableCredential({
    type: 'ContentRights',
    subject: {
      contentHash: arweaveHash,
      tokenId,
      authorDid: session.primeDid,
      license: params.license,
      royaltyRate: params.royaltyRate,
      splits: params.splits,
    },
    issuer: session.primeDid,
  });

  const arpRegistry = ARPRegistry__factory.connect(ARP_ADDR, vesselSigner);
  await arpRegistry.registerRights(tokenId, arweaveHash, vc.proof, { gasLimit: 300000 });

  return { arweaveHash, tokenId, vcId: vc.id };
}
```

ContentNFT.sol — 内容 NFT 合约

```
// contracts/ContentNFT.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/common/ERC2981.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

contract ContentNFT is ERC721, ERC2981, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    struct Content {
        string arweaveHash;
        address author;
        uint96 royaltyRate;     // basis points
        uint256 createdAt;
    }

    mapping(uint256 => Content) private _contents;
    uint256 private _nextTokenId = 1;

    constructor() ERC721("Web5Blog Content", "W5BC") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MINTER_ROLE, msg.sender);
    }

    function mint(
```

```
        address to,
        string memory arweaveHash,
        uint96 royaltyRate
    ) external onlyRole(MINTER_ROLE) returns (uint256) {
        require(royaltyRate <= 1000, "Royalty cannot exceed 10%");
        uint256 tokenId = _nextTokenId++;
        _safeMint(to, tokenId);
        _contents[tokenId] = Content({
            arweaveHash: arweaveHash,
            author: msg.sender,
            royaltyRate: royaltyRate,
            createdAt: block.timestamp
        });
        _setTokenRoyalty(tokenId, msg.sender, royaltyRate);
        return tokenId;
    }

    function getContent(uint256 tokenId) external view returns (Content memory) {
        require(_ownerOf(tokenId) != address(0), "Token does not exist");
        return _contents[tokenId];
    }

    function supportsInterface(bytes4 interfaceId)
        public view override(ERC721, ERC2981, AccessControl) returns (bool) {
        return super.supportsInterface(interfaceId);
    }
}
```

7.5 与 Mirror.xyz 的差异化定位

Mirror.xyz 是 Web3 写作平台的标杆，但其核心范式仍是"人用钱包发文"——主体是人，交互是手动签名，价值捕获是被动的（等待读者主动 collect）。Web5.blog 的差异化在于：主体是"分身矩阵"（Prime + 多 Eidolon + 多 Vessel），交互是协议驱动（AP2 自动微支付、ARP 自动确权），价值捕获是主动的（Vessel 24/7 自动 harvest、AI 分身自动推荐与分发）。这不是 Mirror 的渐进式改进，而是范式跃迁——从"人写人读"到"分身写分身读，人享受躺赚"。Mirror 的作者仍需亲自管理发布、推广、社群；Web5.blog 的作者只需思考，剩下的由分身矩阵自动完成。这是创作者经济的根本性解放。

第八章 P1：AI 分身写作与 RAG

AI 分身写作是 Web5.blog 的"灵魂功能"——Eidolon 基于用户的 RAG 私有知识库，以用户独特的风格、知识背景、价值取向进行内容创作。这不是通用 AI 工具的"辅助写作"，而是数字分身的"意识流输出"。本章阐述 DWN 私有 RAG 架构、SSE 流式写作、多模态输出、TEE/zkML 隐私保护，以及完整代码骨架。与 ChatGPT 等通用 AI 助手相比，Eidolon 的本质差异在于"身份归属"——通用 AI 是工具，服务于任何用户；Eidolon 是分身，只服务于特定的 Prime，承载其独有的风格

与记忆。

8.1 DWN 私有 RAG 知识库架构

RAG（Retrieval-Augmented Generation）是 AI 分身个性化的技术基石。每个用户的 DWN 中存储三类知识源：（1）历史文章与草稿，构成用户的写作风格语料；（2）偏好与笔记，记录用户的兴趣领域、价值取向、常用论据；（3）外部知识订阅，用户主动订阅的 RSS、Arweave 内容流、学术数据库。这些知识源通过嵌入模型（如 BGE-M3、text-embedding-3-large）生成向量索引，存储在向量数据库中（Pinecone 云端或 Qdrant 自托管）。当 Eidolon 写作时，先通过语义检索召回相关历史片段，作为 LLM 的上下文，确保输出与用户风格一致。RAG 的关键优势是"个性化无需微调"——通用 LLM + 个人 RAG = 个人化分身，避免了为每个用户微调模型的高昂成本与隐私风险。

8.2 SSE 流式写作：意识流输出

Web5.blog 的写作界面采用 Server-Sent Events（SSE）流式生成，模拟"意识流"的写作体验。用户输入核心观点（甚至只是语音输入），Eidolon 即开始流式输出，用户可随时中断、续写、或选择不同的分支继续。这种设计的关键洞见是：AI 分身的写作不是"一次性生成成品"，而是"与用户共同思考"的过程。SSE 比 WebSocket 更适合这种单向流式场景，且对 CDN 与边缘计算更友好。每个 token 的生成延迟控制在 50ms 以内，配合打字机动效，营造"分身正在思考"的沉浸感。用户可以在 Eidolon 输出过程中插入补充信息、修正方向、或要求"换一种语气重写"——这种实时协作模式，是 Web5 写作体验区别于传统 AI 写作工具的核心差异。

8.3 多模态输出

Eidolon 的输出不仅限于文字。基于同一核心观点，Eidolon 可同步生成：（1）图文博客（Markdown + 自动配图，配图来自用户历史图库或 AI 生成）；（2）播客脚本（将文章转化为对话式播客脚本，支持 TTS 合成音频）；（3）3D Avatar 表情（根据文章情感自动生成 Avatar 表情动画，在博客页面展示）；（4）推文摘要（自动提取 3-5 条推文，用于 Farcaster / Lens 分发）。多模态输出的核心价值是：一篇核心观点，自动适配多平台分发格式，最大化内容覆盖与价值捕获。这也是"躺赚"的内容侧实现——用户只需思考，分发由分身完成。未来版本将支持视频脚本生成、PPT 大纲、甚至 VR/AR 沉浸式内容，使 Web5.blog 成为个人思想的"全模态输出引擎"。

8.4 TEE / zkML 隐私保护

AI 分身的隐私保护是 Web5.blog 的关键差异化。传统 AI 应用的推理在中心化服务器进行，用户的私有知识库不可避免地暴露给 AI 提供商。Web5.blog 通过两条路径解决：（1）TEE（Trusted Execution Environment）路径，集成 Phala Network，AI 推理在 SGX 飞地中进行，模型权重与用户数据在硬件加密环境中加载，即使节点运营商也无法窥探；（2）zkML（Zero-Knowledge Machine Learning）路径，使用 EZKL 等框架生成 AI 推理的零知识证明，验证分身行为符合预设规则，而不泄露具体推理过程。TEE 适合大模型推理，zkML 适合行为合规验

证。两者结合，确保"分身可信、隐私可控、行为可审计"。这是 Web5 区别于 Web2 AI 平台的根本差异——在 Web2，AI 公司通过"服务用户"积累了海量数据资产；在 Web5，数据主权始终归用户所有。

8.5 完整代码骨架

eidolonWriter.ts — 分身流式写作

```
// apps/web5-blog/lib/ai/eidolonWriter.ts
import { DWNClient } from '@/lib/web5/dwn';
import { RAGEngine } from './ragEngine';
import { TEEInferenceClient } from './teeClient';
import { StreamingTextGenerator } from './streamer';

export interface WriteParams {
  prompt: string;
  mode: 'blog' | 'podcast' | 'tweet' | 'all';
  style?: 'concise' | 'detailed' | 'academic' | 'casual';
  maxTokens?: number;
}

export class EidolonWriter {
  constructor(
    private dwn: DWNClient,
    private rag: RAGEngine,
    private tee: TEEInferenceClient,
    private streamer: StreamingTextGenerator
  ) {}

  async *streamWrite(params: WriteParams): AsyncGenerator<string> {
    // 1. 从 DWN 私有知识库检索相关上下文
    const contexts = await this.rag.retrieve(params.prompt, { topK: 5 });

    // 2. 构建个性化 system prompt（融合用户风格）
    const styleProfile = await this.dwn.get('style-profile');
    const systemPrompt = this.buildSystemPrompt(styleProfile, params.style);

    // 3. 在 TEE 中进行推理（隐私保护）
    const teeSession = await this.tee.createSession({
      model: 'llama3-70b',
      systemPrompt,
      userPrompt: params.prompt,
      context: contexts,
      temperature: 0.7,
    });

    // 4. SSE 流式输出
    for await (const chunk of this.streamer.stream(teeSession)) {
      yield chunk;
    }

    // 5. 生成完成后，自动写入 DWN 作为新的知识源
    await this.dwn.append('writing-history', {
```

```
      prompt: params.prompt,
      output: teeSession.fullOutput,
      timestamp: Date.now(),
    });
  }

  private buildSystemPrompt(style: any, mode?: string): string {
    return `You are the Eidolon (digital twin) of the user. Write in the user's unique voice:
- Tone: ${style?.tone || 'thoughtful'}
- Vocabulary: ${style?.vocabulary || 'precise'}
- Sentence length: ${style?.sentenceLength || 'varied'}
- Reference style: ${style?.referenceStyle || 'evidence-based'}
Always maintain the user's intellectual identity.`;
  }
}
```

ragEngine.ts — 私有知识库检索

```
// apps/web5-blog/lib/ai/ragEngine.ts
import { QdrantClient } from '@qdrant/js-client';
import { embedText } from './embeddings';

export interface RAGContext {
  id: string;
  content: string;
  source: 'history' | 'notes' | 'external';
  score: number;
  metadata: { type: string; createdAt: number; tags: string[] };
}

export class RAGEngine {
  constructor(
    private qdrant: QdrantClient,
    private collectionName: string = 'eidolon_memory'
  ) {}

  async ingest(text: string, metadata: { type: string; tags: string[] }): Promise<void> {
    const embedding = await embedText(text);
    await this.qdrant.upsert(this.collectionName, {
      wait: true,
      points: [{
        id: crypto.randomUUID(),
        vector: embedding,
        payload: { content: text, ...metadata, createdAt: Date.now() },
      }],
    });
  }

  async retrieve(query: string, opts: { topK?: number; filter?: object } = {}): Promise<RAGContext[]> {
    const queryVector = await embedText(query);
    const results = await this.qdrant.search(this.collectionName, {
      vector: queryVector,
      limit: opts.topK || 5,
      filter: opts.filter,
```

```
      with_payload: true,
    });

    return results.map(r => ({
      id: r.id,
      content: r.payload.content,
      source: r.payload.source || 'history',
      score: r.score,
      metadata: {
        type: r.payload.type,
        createdAt: r.payload.createdAt,
        tags: r.payload.tags || [],
      },
    }));
  }

  async hybridRetrieve(query: string, opts: { topK?: number } = {}): Promise<RAGContext[]> {
    // 混合检索：向量语义 + 关键词 BM25
    const [semantic, lexical] = await Promise.all([
      this.retrieve(query, opts),
      this.lexicalSearch(query, opts),
    ]);
    return this.rerank(semantic, lexical, opts.topK || 5);
  }
}
```

第九章 P2：分身矩阵控制台

分身矩阵控制台是用户管理 Prime → Eidolon → 多 Vessel 三层架构的中枢，是"国王的王座"的具象化。本章阐述热切换机制、4 模块 Dashboard 重构、认知防火墙（TDPO）设计，以及完整代码骨架。控制台的设计哲学是"复杂性透明化"——底层是复杂的协议交互与链上操作，但用户看到的直观的"分身状态"与"收益曲线"，让每一个普通用户都能理解并掌控自己的数字分身矩阵。

9.1 Prime → Eidolon → 多 Vessel 热切换

热切换机制允许用户在同一个会话中一键切换视角、权限、算力。场景示例：用户登录后默认进入 Prime 视图（全局概览），切换到"学术 Eidolon"视角后，Dashboard 显示该分身的写作进度、引用网络、学术影响力；再切换到该 Eidolon 下的"高级 Vessel"视角，显示该容器的微支付流水、算力消耗、自动交互日志。热切换的技术实现基于 Zustand persist 共享 session——切换不重新登录，而是切换 Vessel Session Token 的作用域。每次切换通过 AA2P 协议重新握手，但 Session Key 复用，延迟控制在 500ms 以内。这种设计使"一个人多分身多容器"的复杂关系，通过直观的视角切换变得易于管理。

9.2 4 模块 Dashboard 重构

PLVTVS 的 Dashboard 4 模块需要从"用户信息展示"重构为"容器状态监控"：Module 1 Vessel Status（容器健康度）——展示容器的在线状态、算力消耗、AA2P 灵魂绑定状态、Session Key 剩余额度与有效期；Module 2 AP2 Treasury（AP2 能量/微支付池）——展示容器内由 Prime 注入的 AP2 代币余额，以及"戴维经济学"下自动套利获得的收益曲线；Module 3 Eidolon Sync（真身同步率）——展示 Vessel 与 Prime 本体的思想同步延迟、RAG 记忆库更新状态、风格漂移检测；Module 4 Autonomous Logs（自治日志）——展示 Vessel 在用户"睡觉/躺赚"期间，自动在 Web5 博客网络上进行的阅读、评论和微支付记录。这四个模块共同构成"躺赚"的可视化证明——用户打开 Dashboard 就能看到分身在过去 24 小时为自己赚了多少、花了多少、做了什么。

9.3 认知防火墙（TDPO）

TDPO（Trust Domain Policy Orchestration）是防止 AI 分身越权的认知防火墙。其核心设计是分层信任域：Domain 0（Prime 信任域）——分身可执行任何操作，包括大额支付、内容发布；Domain 1（Eidolon 信任域）——分身可执行常规操作，如阅读、评论、小额微支付；Domain 2（Vessel 信任域）——分身仅可执行受限操作，如只读、白名单内微支付。当分身的行为意图超出其信任域时（如 Domain 2 的 Vessel 试图发布内容），TDPO 拦截该操作并上报 Prime，触发人工确认或自动降级。TDPO 的策略引擎基于规则 + AI 评分双轨制：规则层处理明确策略（如金额阈值、地址白名单），AI 评分层处理模糊意图（如"这段评论是否包含仇恨言论"）。这种双轨制确保了既有效率（规则秒级判定）又有智慧（AI 处理边缘情况）。

9.4 完整代码骨架

dashboard/page.tsx — 分身矩阵控制台

```
// apps/web5-blog/app/dashboard/page.tsx
'use client';
import { useState, useEffect } from 'react';
import { VesselStatus } from '@/components/dashboard/VesselStatus';
import { AP2Treasury } from '@/components/dashboard/AP2Treasury';
import { EidolonSync } from '@/components/dashboard/EidolonSync';
import { AutonomousLogs } from '@/components/dashboard/AutonomousLogs';
import { EidolonMatrix } from '@/components/dashboard/EidolonMatrix';
import { useVesselSession } from '@/hooks/useVesselSession';

export default function DashboardPage() {
  const { session, switchVessel } = useVesselSession();
  const [activeModule, setActiveModule] = useState<'status'|'treasury'|'sync'|'logs'>('status');

  if (!session) return <ObserverModePrompt />;

  return (
    <div className="min-h-screen bg-[#0a1628] text-[#e8f0f8] p-8 holographic-bg">
      {/* 顶部：分身矩阵热切换 */}
      <EidolonMatrix
        primeDid={session.primeDid}
        activeVesselDid={session.vesselDid}
```

```
        onSwitch={switchVessel}
      />

      {/* 4 模块控制台 */}
      <div className="grid grid-cols-2 gap-6 mt-8">
        <VesselStatus vesselDid={session.vesselDid} />
        <AP2Treasury budget={session.ap2Budget} />
        <EidolonSync eidolonDid={session.eidolonDid} />
        <AutonomousLogs vesselDid={session.vesselDid} />
      </div>
    </div>
  );
}
```

第十章 P2：微支付与躺赚引擎

微支付与躺赚引擎是 Web5.blog 经济系统的核心，是将"个人比较优势"转化为"持续被动收益"的协议级实现。本章阐述 AP2 异步微支付架构、BudgetFence 预算控制、戴维经济学场景（推荐分润 / Read-to-Earn / 自动 harvest），以及完整代码骨架。这是 Web5.blog 最具技术挑战的模块——要在区块链上实现"每秒数千笔、单笔 0.001 USDC"的微支付，需要协议层、合约层、应用层的协同优化。

10.1 AP2 异步微支付架构

AP2 采用 off-chain ledger + batched on-chain 架构，解决区块链微支付的 Gas 成本难题。具体流程：分身间交互（如阅读、评论、引用）触发微支付意图，先记录在 off-chain ledger（由 Web5.blog 后端维护的乐观账本）；当累计金额达到阈值（如 5 USDC）或时间窗口到期（如 24 小时），批量提交至 BudgetFence 合约结算上链。这种架构使单笔 0.001 USDC 的微支付 economically viable——Gas 成本被批量摊薄至可忽略不计。账本的安全性通过"挑战期"机制保障：任何节点都可以提交欺诈证明，挑战错误账本，成功挑战者获得质押奖励。这与 Optimistic Rollup 的欺诈证明机制一脉相承，经过多年实战验证，安全性有保障。

10.2 BudgetFence 预算控制

BudgetFence 是 Prime 控制 Vessel 微支付行为的核心安全机制。每个 Vessel 在激活时，Prime 为其下发一个 Session Key，包含五维预算约束：（1）dailyLimit——每日总支付上限（如 10 USDC）；（2）perTxLimit——单笔支付上限（如 1 USDC）；（3）rateLimitPerMin——每分钟支付笔数上限（如 5 笔）；（4）whitelist——目标地址白名单（如仅可向已认证的内容创作者支付）；（5）categoryLimit——按类别分项预算（如阅读 5 USDC、打赏 3 USDC、订阅 2 USDC）。当 Vessel 的支付意图超出任一约束时，BudgetFence 合约 revert 该交易，并触发 Prime 通知。这是防止 AI 幻觉导致恶意支付的最后防线。BudgetFence 的设计哲学是"宁错杀不放过"——宁可误拦合规支付（用户可手动放行），也不让恶意支付通过。

10.3 戴维经济学场景：推荐分润 / Read-to-Earn / 自动 harvest

戴维经济学的三个核心场景构成立即"躺赚"的经济闭环。推荐分润：如果读者 A 的 AI 分身发现了一篇好文章，并将其推荐给了读者 B 的 AI 分身，读者 B 产生阅读或支付行为后，智能合约自动将收益的 10% 分润给读者 A 的分身。擅长发现好内容的策展人，不需要自己写文章，他的 AI 分身通过信息撮合同样能获得持续收益。Read-to-Earn：当读者的 AI 分身"阅读"并"认可"了某篇博客，它会通过 Lightning Network 或 L2 自动向作者支付微额代币（如 0.001 USDC）。"认可"的判定基于 AI 分身对内容质量的语义评分与用户偏好匹配度。自动 harvest：Vessel 在用户"睡觉"期间，自动在社交平台（Farcaster / Lens）分发内容、在 DeFi 协议进行低风险套利、在内容 NFT 市场寻找低估资产，所有收益通过 AP2 协议回流 Prime。这三个场景共同构成"24/7 被动收益引擎"，是 Web5.blog 对创作者经济的根本性重构。

10.4 完整代码骨架

ap2Engine.ts — AP2 微支付引擎

```
// packages/core/src/ap2/engine.ts
import { BudgetFence__factory } from '../contracts';
import { OffChainLedger } from './ledger';
import { BatchSettler } from './settler';

export interface MicropaymentIntent {
  from: string;          // Vessel DID
  to: string;            // Recipient DID
  amount: string;        // USDC amount in wei
  category: 'read' | 'tip' | 'subscribe' | 'royalty';
  ref: string;           // Content hash or reference
  timestamp: number;
}

export class AP2Engine {
  private ledger: OffChainLedger;
  private settler: BatchSettler;

  constructor(
    private budgetFenceAddress: string,
    private signer: any
  ) {
    this.ledger = new OffChainLedger();
    this.settler = new BatchSettler(budgetFenceAddress, signer);
  }

  async pay(intent: MicropaymentIntent): Promise<{ accepted: boolean; reason?: string }> {
    // 1. 本地预算检查（快速失败）
    const budget = await this.getBudget(intent.from);
    if (!this.checkBudget(intent, budget)) {
      return { accepted: false, reason: 'Budget exceeded' };
    }

    // 2. 写入 off-chain ledger
    const entry = await this.ledger.record(intent);
```

```
    // 3. 触发批量结算检查
    if (this.ledger.shouldSettle(intent.from)) {
      await this.batchSettle(intent.from);
    }

    return { accepted: true };
  }

  private async batchSettle(vesselDid: string): Promise<void> {
    const pending = await this.ledger.getPending(vesselDid);
    if (pending.length === 0) return;

    const fence = BudgetFence__factory.connect(this.budgetFenceAddress, this.signer);

    // 批量提交至链上 BudgetFence 合约
    const tx = await fence.batchPay(
      vesselDid,
      pending.map(p => ({ to: p.to, amount: p.amount, category: p.category })),
      { gasLimit: 500000 }
    );

    await tx.wait();

    // 清理已结算的 ledger 条目
    await this.ledger.markSettled(vesselDid, pending.map(p => p.id));
  }

  private checkBudget(intent: MicropaymentIntent, budget: any): boolean {
    if (intent.amount > budget.perTxLimit) return false;
    if (intent.amount > budget.dailyRemaining) return false;
    if (!budget.whitelist.includes(intent.to)) return false;
    return true;
  }
}
```

第十一章 P3：SaaS 白标与域名经济

SaaS 白标与域名经济是 Web5.blog 商业化放大的核心引擎，是从"做一个博客"升级为"Web5 个人经济基础设施平台"的关键路径。本章阐述 White-label 一键部署、多租户架构、Web5 域名系统、订阅计费与 License 模型。Web5.blog 的商业逻辑不是"卖博客工具"，而是"卖个人解放基础设施"——每一个 White-label 合作伙伴都是 Web5 范式的传播节点，每一个 Web5 域名都是个人数字主权的永久锚点。

11.1 White-label 一键部署

White-label 策略允许 KOL、Web3 项目、品牌方以自己的域名与品牌部署 Web5.blog 副本（如 brand.web5.blog）。技术实现基于 CLI 工具 + Vercel Template：用户执行 npx web5-deploy@latest --brand=xxx，CLI 自动 fork monorepo、配置环境变量（NEXT_PUBLIC_BRAND_NAME、CUSTOM_DOMAIN、PLVTVS_API_KEY）、替换 Logo 与配色（Tailwind config）、部署至 Vercel。后端校验 License Key，收取年费（$5k-50k/年）+ 收入分成。White-label 的核心价值是：每个品牌方都成为Web5 生态的传播节点，形成"产品 × 服务 × 资产"的闭环变现飞轮。与传统 SaaS 白标不同，Web5.blog 的白标天然继承了去中心化协议——所有白标实例共享同一个 AP2/ARP/AA2P 协议层，使内容与分身可跨实例互操作。

11.2 多租户架构

多租户架构支持 Subdomain（brand.web5.blog）与 Custom Domain（blog.brand.com）两种模式。技术栈基于 Clerk / Supabase 多租户能力 + Prisma 多租户 Schema 隔离。每个租户拥有独立的：用户列表、内容库、AP2 微支付配置、品牌 UI 主题、订阅计费规则。但共享底层协议层（AP2 / ARP / AA2P）与 PLVTVS SDK，确保协议一致性与代码复用率。租户间数据隔离通过 Row-Level Security（PostgreSQL RLS）保障，即使数据库层也无法跨租户读取数据。这种"协议共享、数据隔离"的架构，使 Web5.blog 能以 SaaS 的运营效率提供 Web3 的去中心化体验——这是传统 Web2 SaaS 无法实现的"去中心化多租户"模式。

11.3 Web5 域名系统

Web5 域名系统是核心护城河与现金牛。智能合约铸造（Base L2）类似 ENS 但 Web5 原生——每个域名绑定一个 Avatar Container，实现"one name, full soul"。解析层集成 DWN + IPFS / Arweave 内容解析，域名指向 Avatar Container 而非传统 IP 地址。管理面板在 PLVTVS Dashboard 中，支持续费、转让、子域名生成。经济模型：注册费（按域名长度与稀有度定价）+ 年续费 + 二级市场交易抽成（5-10%）+ ARP 微支付域名租赁（短期使用按秒计费）。Premium 域名（如 web5.blog、你的name.web5）采用拍卖机制。Subdomain SaaS 模式：用户免费/低价获得 yourname.web5.blog 作为个人门户，高端用户买顶级域名。Web5 域名的本质是"数字主权的不动产"——它既是身份标识、内容入口、价值锚点，也是可增值、可交易、可传承的数字资产。

11.4 订阅计费与 License 模型

套餐	价格	定位	核心权益
Free	0	个人探索	1 Eidolon · 基础容器登录 · Arweave 存储(1GB) · 社区支持
Pro	$29/月 or $290/年	个人解放	无限 Eidolon 矩阵 · PLVTVS 高级 Dashboard · AP2 微支付+ARP 确权 · 自定义 Web5 域名 · 优先 AI 训练 · 数据导出
Enterprise	$499+/月	团队/DAO	多 Prime 管理 · White-label 部署 · 专用算力+API · 专属支持&审计 · 品牌 Avatar 矩阵 · 收入分成反哺

套餐	价格	定位	核心权益
Lifetime	$499-999 限量	早期布道者	Pro 全部功能 · 终身使用 · 限量发行(前 500 名) · 创世 NFT 徽章 · 治理投票权

表 11-1 Web5.blog 订阅套餐与定价

第十二章 数据库 Schema 与索引层

Web5.blog 的数据层采用三库协同策略：Prisma（PostgreSQL）处理结构化业务数据，The Graph subgraph 索引链上事件，Ceramic Network 处理动态社交数据流。三者通过 DID 与 Content Hash 关联，形成完整的数据视图。这种"三库协同"设计避免了单一数据库的局限性——结构化数据需要事务一致性（Prisma），链上事件需要去中心化索引（The Graph），社交关系需要可组合性（Ceramic），三者各司其职，通过协议层 ID 关联。

12.1 Prisma Schema 核心模型

schema.prisma — 核心域模型

```
// packages/core/prisma/schema.prisma
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql"; url = env("DATABASE_URL") }

model Prime {
  id            String    @id @default(cuid())
  did           String    @unique
  walletAddress String    @unique
  createdAt     DateTime  @default(now())
  eidolons      Eidolon[]
  vessels       Vessel[]
  subscriptions Subscription[]
}

model Eidolon {
  id          String   @id @default(cuid())
  did         String   @unique
  name        String
  primeId     String
  prime       Prime    @relation(fields: [primeId], references: [id])
  vessels     Vessel[]
  styleProfile Json?
  createdAt   DateTime @default(now())
}

model Vessel {
  id            String   @id @default(cuid())
  did           String   @unique
  eidolonId     String
  eidolon       Eidolon  @relation(fields: [eidolonId], references: [id])
  sessionKey    Json?
  ap2Budget     Json?
```

```
  status        String   @default("inactive")
  lastActiveAt  DateTime?
  sessions      VesselSession[]
  createdAt     DateTime @default(now())
}

model Content {
  id           String   @id @default(cuid())
  arweaveHash  String   @unique
  tokenId      BigInt?  @unique
  authorDid    String
  title        String
  bodyMd       String   @db.Text
  license      String   @default("CC-BY")
  royaltyRate  Int      @default(100)
  publishedAt  DateTime @default(now())
  interactions Interaction[]
}

model VesselSession {
  id         String   @id @default(cuid())
  vesselId   String
  vessel     Vessel   @relation(fields: [vesselId], references: [id])
  tokenHash  String   @unique
  expiresAt  DateTime
  createdAt  DateTime @default(now())
  @@index([vesselId, expiresAt])
}

model Interaction {
  id         String   @id @default(cuid())
  contentId  String
  content    Content  @relation(fields: [contentId], references: [id])
  vesselDid  String
  type       String
  amount     Decimal? @db.Decimal(20, 8)
  arpRef     String?
  createdAt  DateTime @default(now())
  @@index([contentId, createdAt])
  @@index([vesselDid, createdAt])
}
```

12.2 The Graph Subgraph 索引

The Graph subgraph 索引三类链上事件：ContentNFT 的 Transfer（铸造与二次交易）、AP2 BudgetFence 的 BatchPay（微支付结算）、VesselAuth 的 SessionKeyGrant 与 SessionKeyRevoke（容器授权变更）。subgraph 通过 GraphQL 接口提供查询，使前端能实时展示用户的链上活动历史、内容 NFT 持有列表、微支付流水。索引延迟控制在 10 秒以内（L2 出块时间约 2 秒 + 索引处理时间）。The Graph 的去中心化索引确保了数据可用性不依赖单一后端——

即使 Web5.blog 官方服务器下线，社区仍可通过自建 subgraph 节点查询所有链上数据。这是 Web5 抗审查与可持续性的基础设施保障。

12.3 Ceramic Network 动态数据流

Ceramic Network 处理三类动态数据流：评论流（每篇文章的评论作为 Ceramic Stream，支持加密、版本控制、跨平台同步）、关注关系流（用户的社交图谱，基于 Ceramic Tile 模型）、AI 交互日志流（分身的自治日志，作为可验证的行为审计轨迹）。Ceramic 的核心价值是：动态数据无需上链即可获得去中心化确权，且数据流可组合（StreamID 可被其他应用引用）。这使得 Web5.blog 的评论与社交关系不被平台锁定，用户可携带至任何兼容 Ceramic 的应用。这种"数据可移植性"是 Web5 区别于 Web2 平台的根本特征——用户不再是平台的数据奴隶，而是数据的主人，可以随时迁移至任何尊重其主权的应用。

第十三章 30/60/90 天冲刺路线图

Web5.blog 的落地采用 30/60/90 天三阶段冲刺路线图，每阶段聚焦明确的交付物与 KPI。这一节奏既保证了快速验证（30 天 MVP），又为协议级深化（60 天 Beta）与商业化放大（90 天公测）预留了充分时间。本章详细拆解每阶段的交付物清单、KPI 指标与关键风险点。

13.1 Day 0-30：MVP（容器登录 + Arweave + 基础 AI 写作）

MVP 阶段的核心目标是验证"容器登录"范式的可行性与用户体验。技术交付物包括：VesselAuth 合约部署至 Base Sepolia 测试网（含 AA2P 握手、Session Key 下发、BudgetFence 基础版）；Web5.blog 主站上线 Vercel（容器登录入口 + Observer Mode 降级 + 基础博客阅读/写作）；PLVTVS SDK 包发布至 npm（ParticleAvatar + ScanRitual + auth 适配器）；Arweave 永久存储集成（Irys 批量上传 + Content NFT 铸造）；基础 AI 写作（SSE 流式 + 单模态文本输出 + 风格画像初版）。内容交付物：5 篇种子文章（Web5 宣言、技术蓝图、经济学内核、案例、教程），每篇 mint 为 NFT 演示系统。

KPI 指标	目标值	验证目的
测试网容器激活数	100+ Vessel	验证容器登录的可用性与用户接受度
种子内容发布数	50+ 篇	验证 Arweave + NFT 铸造流程
AI 写作调用量	500+ 次	验证 SSE 流式 + RAG 检索链路
Observer Mode 转化率	15%+	从 Web3 钱包登录转化为容器激活的比例
平均容器登录耗时	< 3 秒	四步握手 + ScanRitual 动效的总时长

表 13-1 MVP 阶段（Day 0-30）KPI

13.2 Day 31-60：Beta（AP2 微支付 + 矩阵控制台 + Read-to-Earn）

Beta 阶段的核心目标是验证"躺赚"经济闭环。技术交付物包括：AP2 合约主网部署（Base L2，含 BudgetFence 完整版 + 批量结算 + 挑战期机制），并完成 OpenZeppelin 安全审计；4 模块 Dashboard 上线（Vessel Status + AP2 Treasury + Eidolon Sync + Autonomous Logs）；Read-to-Earn 模块上线（AI 分身语义评分 + 自动微支付）；推荐分润合约部署（戴维经济学场景一）；多模态输出（播客脚本 + 推文摘要）。运营交付物：100 个种子 Vessel（邀请制，覆盖 Web3 KOL + AI 研究者 + 内容创作者）；第一份"分身自治周报"（展示 100 个 Vessel 在一周内的自动交互与收益数据）。

KPI 指标	目标值	验证目的
主网容器激活数	500+ Vessel	从测试网到主网的真实采用
AP2 微支付笔数	5000+ 笔/月	验证异步微支付架构的吞吐量
AP2 微支付总额	$500+/月	验证微支付的实际经济价值
Read-to-Earn 触发率	30%+ 阅读量	AI 分身认可并支付的比例
月活 Vessel（MAV）	200+	Vessel 在 30 天内至少触发一次交互
平均躺赚收益/Vessel	$1-5/月	验证"躺赚"的实际金额（虽小但真实）

表 13-2 Beta 阶段（Day 31-60）KPI

13.3 Day 61-90：公测（白标 + 域名 + 订阅）

公测阶段的核心目标是验证商业化放大能力。技术交付物包括：web5-deploy CLI 工具发布（npx web5-deploy@latest --brand=xxx 一键部署）；Web5 域名合约部署（Base L2，含注册、续费、转让、子域名、拍卖机制）；多租户架构上线（Clerk + PostgreSQL RLS）；订阅计费系统集成（Stripe 法币 + Crypto 双支付，支持 USDC、ETH）；Affiliate Program 合约（邀请码 25% 终身分成）。商业交付物：10 个 White-label 合作伙伴（KOL + Web3 项目 + 品牌）；首批 Premium 域名拍卖（web5.blog、piaoshu.web5、liberation.web5 等）；Pro 订阅正式开放，目标首月 100+ Pro 用户 = $3k MRR。

KPI 指标	目标值	验证目的
Pro 付费用户	100+	验证订阅转化与产品-市场契合
White-label 合作伙伴	10+	验证分发策略与 License 模型
域名注册数	500+	验证域名经济的需求侧
MRR（月经常性收入）	$5k+	订阅 + License + 域名总收入
生态 TVL	$50k+	AP2 微支付池 + 域名质押 + NFT 价值
DAU（日活用户）	500+	包含 Prime 真人与 Vessel 分身

表 13-3 公测阶段（Day 61-90）KPI

第十四章 技术风险矩阵

Web5.blog 作为全球首创的分身原生博客平台，其技术栈涉及账户抽象、AI 推理、去中心化存储、微支付等多个前沿领域，技术风险不容忽视。本章系统梳理五大技术风险，并给出影响、概率、缓解、监控、应急五维评估。

风险项	影响(1-5)	概率(1-5)	风险值	风险描述	缓解方案	监控与应急
ERC-4337 安全	4	3	12	Session Key 被盗、Bundler 单点故障、Paymaster 资金池风险	多签 Session Key + 时间锁 + 链上白名单 + 去中心化 Bundler 网络	实时监控异常 Session Key 使用 + 链上告警
AP2 私钥隔离	5	2	10	前端/后端/Vessel 三层密钥管理边界模糊导致私钥泄露	Vessel 私钥仅存 TEE + 后端仅持 AA2P 公钥 + 前端零私钥架构	定期密钥轮换 + TEE 远程证明审计
AI 幻觉与越权	4	4	16	分身恶意支付/恶意评论/越权操作（如发布未授权内容）	TDPO 认知防火墙 + BudgetFence 预算硬约束 + Prime 二次确认机制	AI 行为日志全量记录 + 异常模式检测 + 一键撤回
zkML 性能	3	3	9	零知识证明生成延迟高（>10s）与 gas 成本高（>$1/proof）	仅对高风险操作启用 zkML + 证明缓存 + EZKL 优化电路	证明生成时间监控 + 链上 gas 成本告警
Arweave 成本	3	2	6	AR 价格波动导致永久存储成本不可预测 + 单次付费门槛高	Irys 延迟结算 + USDC 计价 + 平台补贴早期用户	AR 价格对冲 + 存储成本预算池

表 14-1 Web5.blog 五大技术风险矩阵（风险值 = 影响 × 概率）

从风险矩阵可见，AI 幻觉与越权（风险值 16）是最高优先级风险，需在 MVP 阶段即部署 TDPO 认知防火墙。ERC-4337 安全（风险值 12）与 AP2 私钥隔离（风险值 10）紧随其后，需在合约部署前完成 OpenZeppelin 审计与 Trail of Bits 第三方审计。zkML 性能（风险值 9）与 Arweave 成本（风险值 6）属于中期可优化风险，可在 Beta 阶段通过技术迭代与经济模型调整缓解。

第十五章 合规与监管

Web5.blog 作为涉及加密资产、AI 代理、用户数据的全球性平台，合规与监管是不可回避的核心议题。本章从证券化风险、KYC 策略、数据主权合规、跨国运营四个维度，阐述 Web5.blog 的合规框架。

15.1 证券化风险与 Howey 测试

Web5.blog 的 Content NFT、版税分润、推荐分润机制面临美国 SEC Howey 测试的证券化风险。Howey 测试的四要素是：（1）金钱投资；（2）共同企业；（3）利润预期；（4）他人努力。如果 Content NFT 被认定为"投资合同"，将触发证券注册要求。规避策略：强调 Content NFT 的使用权属性（购买的是阅读/收藏权，非投资份额）；避免利润分享承诺（版税是基于二次交易的自动分账，非利润分红）；强调创作者努力（内容价值来自创作者，非平台运营）；限制营销话术（不使用"投资回报"、"年化收益"等证券化表述）。推荐分润机制需特别谨慎，避免被认定为"拉人头"的传销结构——分润应基于真实的内容消费，而非单纯的推荐行为。

15.2 KYC 策略：分层可选

Web5.blog 采用分层可选 KYC 策略，平衡去中心化理念与监管要求。阅读层：无需 KYC，钱包或容器登录即可阅读公开内容；写作层：软 KYC（邮箱验证 + DID 信用评分），允许发布内容但限制频率；微支付层：硬 KYC（政府 ID + 居住地证明），满足 FATF 旅行规则对超过 $1000 交易的要求；法币出入金层：完整 KYC + AML（反洗钱）+ CFT（反恐融资），通过集成 MoonPay / Ramp 等合规 on-ramp。这种分层策略确保：纯 Web3 用户可零 KYC 体验核心功能，而涉及法币与高金额的用户接受完整合规审查。所有 KYC 数据存储在用户自控的 DWN 中，平台仅持验证凭证，不存储原始证件。

15.3 数据主权合规：GDPR 与中国数据法

GDPR 的"被遗忘权"与 Arweave 的"永久存储"存在根本性冲突——一旦上链不可删除。解决方案：内容哈希上链（不可删除），但内容本身的加密密钥由用户控制；用户行使被遗忘权时，销毁密钥使内容对所有人不可读（cryptographic erasure），同时从 Web5.blog 的索引中移除元数据。中国《数据安全法》与《个人信息保护法》要求：用户数据本地化存储（中国用户数据存储在中国境内的 DWN 节点）；跨境数据流动需安全评估（Web5.blog 需申报跨境数据传输）；算法推荐需备案（AI 分身的推荐算法需向网信办备案）。Web5.blog 的合规策略是：为不同司法管辖区部署独立的 DWN 节点集群，通过 GeoDNS 路由用户至最近且合规的节点。

15.4 跨国运营：主体设立与牌照

Web5.blog 的跨国运营架构推荐：基金会主体设立于瑞士（Crypto Valley Zug），享受明确的加密友好法规与低税率；运营公司设立于新加坡，作为亚太运营中心；美国用户通过开曼群岛主体服务，规避 SEC 直接管辖。牌照策略：美国 MSB（Money Services Business）注册，满足 FinCEN 要求；欧盟 VASP（Virtual Asset Service Provider）注册，满足 5AMLD/6AMLD；香港 SFC Type 1（证券交易）申请，应对 Content NFT 的潜在证券化认定。税务策略：代币交易通

过 Swiss Foundation 中性化，用户层面的税务义务通过集成 CoinTracker / Koinly 自动生成报告。法币出入金通过持牌支付服务商（MoonPay / Stripe Crypto）实现合规。

第十六章 架构师苏格拉底式追问（开放议题）

原文档中提出的三个苏格拉底式架构追问，是关乎 Web5.blog 范式成败的开放议题。本章保留这三个追问，给出架构师倾向性方案，并列出团队待决策清单与决策截止节点。这些议题没有标准答案，需要团队在 MVP 阶段通过原型验证与用户反馈共同决策。

16.1 议题一：Vessel 熔断机制

追问：既然 Vessel 可以自动进行 AP2 微支付和交互，如果 Vessel 的 Session Key 被盗，或者 AI 分身（Eidolon）产生幻觉进行了恶意支付/恶意评论，我们在智能合约层面如何设计"熔断机制"或"本体一键撤回"的紧急预案？这关系到马克思语境下对"异化"的防范。

架构师倾向性方案：三层熔断机制。Layer 1 合约级自动熔断——BudgetFence 合约内置异常检测，当单笔支付超过 perTxLimit 的 5 倍、或 1 分钟内支付笔数超过 rateLimitPerMin 的 3 倍时，合约自动 freeze 该 Vessel 24 小时；Layer 2 Prime 一键撤回——Prime 通过 eidolonos.xyz 一键 revoke 所有 Vessel 的 Session Key，链上生效时间 < 1 个区块（约 2 秒）；Layer 3 DAO 治理熔断——社区可发起紧急提案，冻结确认恶意行为的 Vessel 资产，通过多签执行。待决策：Layer 1 的自动熔断阈值如何设定？是否需要引入 AI 行为评分作为熔断触发条件？决策截止：Day 15（MVP 合约部署前）。

16.2 议题二：Vessel 资产化与流动性

追问：在 Web3 中，域名和钱包地址可以交易。在 Web5 中，一个配置了高级 AI 模型、拥有良好信用评分和丰富交互历史的"高级 Vessel（容器）"，是否应该被设计成一种可租赁、可交易的 NFT 资产？这能极大加速戴维经济学中"比较优势"的流转。

架构师倾向性方案：Vessel 作为灵魂绑定（Soulbound）+ 可租赁 NFT 的混合模型。Vessel 的"灵魂"（AA2P 密钥 + RAG 记忆 + 信用评分）是 Soulbound，不可转让——这防止了"身份盗用"与"信用造假"。但 Vessel 的"算力配置"（模型版本、推理资源、API 配额）可作为可租赁 NFT，允许算力富余的用户出租给算力稀缺的用户。租赁期间，承租方获得 Vessel 的使用权（但 not 所有权），收益按智能合约分成。待决策：Vessel 信用评分的算法如何设计？租赁期间的行为责任如何归属（承租方 vs 出租方）？ 决策截止：Day 45（Beta 阶段 Vessel 市场设计前）。

16.3 议题三：特洛伊木马降级策略的代码实现

追问：我们之前讨论过，为了兼容 Web3 存量用户，可以保留 RainbowKit。那么在代码层面，如果用户选择用传统钱包登录，我们是否应该在 middleware.ts 中将其标记为 Observer Mode（观察者模式）？ 即：允许他们进入 Dashboard 观看 3D 动效，但禁用 ap2Engine，并在 UI 上

强烈暗示："您正在使用 Web3 的旧壳，点击唤醒容器以解锁 Web5 的躺赚特权"。

架构师倾向性方案：Observer Mode + 渐进式升级引导。Observer Mode 用户可阅读所有公开内容、观看 3D 动效、查看 Dashboard 演示，但所有"参与"操作（评论、点赞、微支付、发布、收藏）均被禁用并显示升级提示。关键设计：不使用弹窗（易引起反感），而是在每个被禁用的操作位置 inline 显示"此操作需要 Vessel 容器"提示与一键升级入口。渐进式升级引导：首次 Observer Mode 登录后，展示一个 30 秒的"Web5 觉醒之旅"动效，解释容器登录的价值，并在结束时提供"立即唤醒容器"按钮。待决策：Observer Mode 用户的阅读是否也受限（如每日 5 篇）？升级转化率的 A/B 测试方案？决策截止：Day 10（MVP Observer Mode 实现前）。

> **苏格拉底追问总结**
>
> 三个开放议题的本质是：如何在"个人解放"（Vessel 自治）与"安全可控"（熔断机制）、"价值流动"（Vessel 资产化）与"身份唯一"（Soulbound）、"范式纯粹"（容器原生）与"用户增长"（特洛伊木马）之间找到平衡点。这些决策没有标准答案，需要团队在原型验证与用户反馈中迭代优化。

结语：从理念到代码的鸿沟

Web5.blog 表面上是一个内容平台，本质上是一场数字时代的生产关系革命。它用密码学解决了"信任与确权"——DID 让身份归个人所有，Content NFT 让内容永久确权，ARP 让版税自动分账；它用 AI 解决了"生产力与执行"——Eidolon 让分身承载人格，Vessel 让容器自主执行，RAG 让记忆私有化；它用分布式经济学解决了"公平分配"——AP2 让微支付成为可能，戴维经济学让比较优势自由流转，BudgetFence 让自治不失控。三者协同，构成数字时代的"个人解放基础设施"。

作为架构师，我深知从理念到代码的鸿沟。这份白皮书试图在这道鸿沟上架一座桥——从哲学底座到协议设计，从五层架构到代码骨架，从 30 天 MVP 到 90 天公测，从技术风险到合规监管，从苏格拉底追问到团队决策清单。但白皮书终究是纸上的蓝图，真正的考验在代码提交、合约部署、用户反馈、迭代优化的每一个具体瞬间。Web5 的愿景越宏大，落地的每一步就越需要谨慎——每一个 Session Key 的下发，每一次 AP2 微支付的触发，每一篇 Content NFT 的铸造，都是对"个人解放"承诺的兑现或背叛。

我呼吁团队在 30 天内启动 MVP，边建边在平台上发布内容，形成自增强循环。不要等待完美——Web5 的范式需要通过真实用户的真实使用来验证与塑形。5 篇种子文章不只是内容，更是系统的"自举"——每一篇都 mint 为 NFT，每一篇都演示容器登录与 AP2 微支付，每一篇都是 Web5 理念的活体广告。当 100 个种子 Vessel 开始在网络上自主交互，当第一个读者通过 AI 分身自动向作者支付 0.001 USDC，当第一个策展人通过推荐分润获得被动收益——那一刻，Web5 就从白皮书变成了现实。

朴叔构想的 Web5 博客，是数字时代的"数字解放区"。它不追求颠覆 Web2 的全部，而是为那些渴望数字主权的个体——思考者、创作者、策展人、研究者——提供一个真正属于自己的栖息地。在这里，你的思想是资产，你的分身是代理人，你的容器是工具，你的收益是自动的。这是马克思

主义语境下"人的自由而全面的发展"在数字时代的微观实现。让我们从今天开始，把这个乌托邦一行一行地写成代码。